

MAT 3510 — Homework 2 Study Notes

Gaussian Elimination and LU Decomposition

These notes cover the *math* behind the problem, some guiding questions, and MATLAB syntax reference. Per the course AI policy, they deliberately do **not** lay out the algorithm step by step in code — turning the math into loops is the part you do yourself.

1. The big picture

Solving $Ax = b$ by Gaussian elimination secretly does two things at once: it turns A into an upper-triangular matrix U , and it records every row operation it used. Recording those operations in a lower-triangular matrix L gives the factorization

$$A = LU.$$

Once you have it, $Ax = b$ becomes $L(Ux) = b$, which splits into two *easy* triangular problems:

$$\underbrace{Lz = b}_{\text{forward substitution}} \quad \text{then} \quad \underbrace{Ux = z}_{\text{backward substitution}}.$$

Why bother factoring instead of just eliminating? Factoring costs about $\frac{2}{3}d^3$ flops; each triangular solve costs only about d^2 . If you must solve with the same A and many different b 's, you pay the expensive part once.

2. What L and U look like

- U is upper triangular: it is exactly the matrix you are left with after elimination zeros out everything below the diagonal.
- L is *unit* lower triangular: ones on the diagonal, zeros above it.
- The entries of L below the diagonal are the **multipliers**. If, to zero out entry (i, k) , you did

$$\text{row}_i \leftarrow \text{row}_i - \ell_{ik} \text{row}_k, \quad \ell_{ik} = \frac{a_{ik}}{a_{kk}},$$

then that number ℓ_{ik} is stored as $L(i, k)$. (Here a_{ik} and a_{kk} mean the entries *at that moment* of the elimination, not the original entries of A .)

A tiny example by hand (2×2)

$$A = \begin{pmatrix} 2 & 1 \\ 6 & 5 \end{pmatrix}.$$

To kill the 6, subtract $3 \times$ row 1 from row 2 (multiplier $\ell_{21} = 6/2 = 3$):

$$U = \begin{pmatrix} 2 & 1 \\ 0 & 2 \end{pmatrix}, \quad L = \begin{pmatrix} 1 & 0 \\ 3 & 1 \end{pmatrix}, \quad LU = \begin{pmatrix} 2 & 1 \\ 6 & 5 \end{pmatrix} = A. \quad \checkmark$$

Try a 3×3 by hand before coding. Notice which multipliers you compute in which order — that order *is* your loop structure.

3. Triangular solves

For an upper-triangular system $Ux = z$, look at the **last** equation first: it involves only one unknown,

$$u_{dd} x_d = z_d.$$

Once x_d is known, the second-to-last equation has only one *unknown* left. Write out the general i -th equation of $Ux = z$ and ask: which x_j 's are already known when I reach row i , and how do I isolate x_i ? Forward substitution for $Lz = b$ is the mirror image, starting from the *first* row. (Bonus: since L has ones on its diagonal, what division do you get to skip?)

4. Guiding questions for designing your function

1. **Inputs/outputs.** What should your function take in and return? Just x ? Or L and U too (useful for part (2) and for checking)? Consider separate functions for the factorization and for each triangular solve.
2. **Columns.** Elimination works column by column. For a $d \times d$ matrix, which columns need entries eliminated below the diagonal? (Does the last column need any work?)
3. **Rows.** For a fixed column k , which rows i get a multiplier? What is the range of i ?
4. **Which entries change?** When you update row i using row k , which columns of that row actually change? Can you do the update with a single vectorized row operation instead of an inner loop?
5. **Zero pivots.** What happens to $\ell_{ik} = a_{ik}/a_{kk}$ if $a_{kk} = 0$? Your function should work for a “general” d , so at minimum decide what it should do in that case (detect it and **error**, or swap rows — partial pivoting, which gives $PA = LU$). For the given test system, work out the pivots by hand: do you ever hit a zero?
6. **Counting.** Your loops should cost $O(d^3)$ for the factorization and $O(d^2)$ per triangular solve. Does your loop nesting depth match that?

5. How to check your answer

- **Reconstruction:** $\text{norm}(L*U - A)$ should be at round-off level (around 10^{-15}).
- **Structure:** is your L really unit lower triangular and U really upper triangular? (`istril`, `istriu` test this.)
- **Residual:** $\text{norm}(A*x - b)$ should be tiny.
- **Against MATLAB:** compare your x to $A \backslash b$. *Careful:* MATLAB's built-in $[L,U,P] = \text{lu}(A)$ uses partial pivoting, so its L and U may differ from yours even when both are correct. Compare solutions and reconstructions, not factor-by-factor.

6. MATLAB syntax reference

These are general language features, not the algorithm.

```
% A function file: must be saved as myfunc.m (file name = function name)
function [out1, out2] = myfunc(in1, in2)
    ...
end

d = size(A, 1); % number of rows of A
I = eye(d); % d-by-d identity matrix
z = zeros(d, 1); % d-by-1 column vector of zeros

for k = 1:d-1 % loop k = 1, 2, ..., d-1
    ...
end
for i = d:-1:1 % loop counting DOWN: d, d-1, ..., 1
    ...
end

A(i, :) % entire row i
A(i, k:end) % row i, columns k through the last
A(k+1:end, k) % column k, rows below the diagonal
v(j+1:end)' * w(j+1:end) % dot product of two sub-vectors

error('Zero pivot at step %d', k); % stop with a message
A = [1 -2 3; 0 1 -1; 1 3 0]; % matrix literal: ; separates rows
b = [4; -3; 1]; % COLUMN vector (note the semicolons)
format long % show more digits when printing
```

7. Common pitfalls

- **Row vs. column b .** $[4 \ -3 \ 1 \ -3]$ is a row vector; $[4; -3; 1; -3]$ is a column. Mixing them causes dimension errors in $A*x - b$.
- **Using stale entries.** The multiplier uses the *current* value of a_{ik} , after earlier eliminations have modified it — not the original A .

- **Off-by-one ranges.** Most bugs in this problem are loop bounds. Trace your code by hand on the 2×2 example above and on a 3×3 you've done on paper.
- **File names.** Each function you call from your test script must live in its own `.m` file with the matching name (same lesson as HW1).